

Solana Bridge Program for JCK and JCKP Tokens README.md

Solana Bridge Program for JCK and JCKP Tokens

License MIT

A Solana-based bridge solution for transferring assets between JCK and JCKP tokens, built on Raydium AMM.

Overview

This project implements a secure bridge for transferring between JCK and JCKP tokens on the Solana blockchain. The bridge leverages Raydium's AMM (Automated Market Maker) to provide liquidity and ensure fair exchange rates between tokens.

The bridge is built on top of the Rarimo bridge infrastructure, which consists of the following components:

- [Bridge](#) - Core program that handles token deposits, swaps, and withdrawals
- [Commission](#) - Program that manages fee collection and distribution
- [Lib](#) - Shared library of utilities used by both bridge and commission programs

Key Features

- **Token Bridging:** Seamlessly bridge between JCK and JCKP tokens
- **Raydium Integration:** Utilize Raydium's liquidity pools for efficient token swaps
- **Security:** Multiple security features including bridge admin controls and pause functionality
- **Fee Management:** Configurable fee structure to ensure bridge sustainability
- **Monitoring:** Tools for bridge health monitoring and maintenance

Prerequisites

Before getting started, ensure you have the following installed:

- [Node.js](#) (v14 or later)
- [Rust](#) and Cargo
- [Solana CLI](#) (v1.10.0 or later)
- [Git](#)

You will also need:

- A Solana wallet with SOL for deployment and testing
- Devnet/Mainnet RPC endpoint access

Installation

1. Clone the repository:

```
git clone https://github.com/your-username/solana-bridge-program.git
cd solana-bridge-program
```

2. Install dependencies:

```
npm install
```

3. Build the bridge programs:

```
npm run build:bridge
npm run build:commission
npm run build:upgrade
```

Configuration

Create a `.env` file in the project root with the following configuration:

```
# Solana Configuration
SOLANA_NETWORK=devnet
SOLANA_RPC_URL=https://api.devnet.solana.com

# Wallet Configuration
WALLET_PRIVATE_KEY=your_private_key_here

# Bridge Configuration
BRIDGE_PROGRAM_ID=your_deployed_bridge_program_id
COMMISSION_PROGRAM_ID=your_deployed_commission_program_id

# Token Configuration
JCK_TOKEN_MINT=your_jck_token_mint_address
JCKP_TOKEN_MINT=your_jckp_token_mint_address
```

Deployment

1. **Generate Program Keypairs** (if not already created):

```
solana-keygen new -o ./dist/program/bridge-keypair.json
solana-keygen new -o ./dist/program/commission-keypair.json
solana-keygen new -o ./dist/program/upgrade-keypair.json
```

2. Deploy the Bridge Programs:

```
solana program deploy --program-id ./dist/program/bridge-keypair.json  
./dist/program/bridge.so  
solana program deploy --program-id ./dist/program/commission-keypair.json  
./dist/program/commission.so  
solana program deploy --program-id ./dist/program/upgrade-keypair.json  
./dist/program/upgrade.so
```

Setup Process

Once the programs are deployed, follow these steps to set up the bridge:

1. Create JCK and JCKP Tokens (if you don't have them already):

```
node create-tokens.js
```

2. Initialize the Bridge Admin:

```
node initialize-bridge.js
```

3. Create Raydium Pool:

```
npm install @raydium-io/raydium-sdk  
node create-raydium-pools.js
```

For detailed setup instructions, refer to the [prompt.txt](#) file.

Usage Flow

The bridge operates with the following flow:

Depositing Tokens

A deposit transaction consists of two instructions:

1. Charge commission instruction to the commission program
2. Deposit instruction (native/ft/nft) to the bridge program

Example for depositing JCK tokens to receive JCKP tokens:

```
const { depositJCK } = require('./bridge-client');  
  
// Deposit 100 JCK tokens  
await depositJCK(100);
```

Example for depositing JCKP tokens to receive JCK tokens:

```
const { depositJCKP } = require('./bridge-client');

// Deposit 100 JCKP tokens
await depositJCKP(100);
```

Withdrawing Tokens

A withdrawal consists of a single transaction with a withdraw instruction.

The instruction requires a signature for the withdrawal data, which is generated by the bridge system.

```
const { withdrawTokens } = require('./bridge-client');

// Withdraw tokens with the provided signature
await withdrawTokens(signature, recoveryId, path, origin, tokenSeed);
```

Checking Transaction Status

```
const { getTransactionStatus } = require('./bridge-client');

// Check the status of a transaction
const status = await getTransactionStatus(transactionId);
console.log(`Transaction status: ${status}`);
```

Bridge Architecture

The bridge consists of the following components:

1. **Bridge Program:** Core Solana program that handles token deposits, swaps, and withdrawals
2. **Commission Program:** Manages fee collection and distribution
3. **Upgrade Program:** Handles program upgrades and migrations
4. **Raydium Integration:** Connects to Raydium AMM for liquidity and token swaps

Monitoring and Maintenance

To monitor the bridge health:

```
node scripts/monitor-bridge.js
```

To perform routine maintenance:

```
node scripts/maintenance.js
```

Security Considerations

- Always use secure private keys and never share them
- Start with small test transactions before committing larger amounts

- Monitor bridge activity regularly for unusual patterns
- Use the emergency pause feature if suspicious activity is detected
- Ensure the commission program is the same as defined in the bridge admin
- Verify the bridge admin is properly configured, otherwise you may lose your tokens

Troubleshooting

Common Issues

Transaction Failure:

- Check if the bridge is paused
- Ensure you have sufficient token balance
- Verify slippage settings

Liquidity Issues:

- Check Raydium pool liquidity
- Temporary high slippage may occur during volatile market conditions

Network Errors:

- Ensure RPC endpoint is responsive
- Check Solana network status

Development

To contribute to the project:

1. Fork the repository
2. Create a feature branch
3. Make your changes
4. Add tests for new functionality
5. Submit a pull request

License

This project is licensed under the [MIT License](#).

Contact

For questions or support, please open an issue on the GitHub repository.